
Alumni Hunt

CS315 Assignment 1 Report

Aditya Khandelwal (210059) Om Shrivastava (210685) Siddharth Garg (211031)
Tejaswa Mathur (211110)

1 Introduction

The assignment focuses on developing a system to manage a structured database of IITK alumni, specifically designed to efficiently handle queries on a dataset with three key fields: *id*, *name*, and *year*.

Data Structure Design

- *Primary Key* (id): A unique 6-digit integer, the alumnus's Aadhar number, as the primary identifier.
- *Name* (name): The name field is a text entry limited to lowercase Latin alphabet characters (a–z) with a minimum of 4 and a maximum of 10 characters.
- *Year of Graduation* (year): This integer field represents the year of graduation, with values constrained between 1900 and 2100. The specific dataset may have a smaller range (e.g., 1964–2024 in public.csv provided).

Our report details the indexing strategies implemented, including the specific indices created, the algorithms used to construct these indices, and the key statistics computed for efficient data access. It also outlines the disk layout strategies applied to optimize storage and retrieval. Additionally, we discuss the query processing methods used, highlighting optimizations that improve overall query performance. Our approach and methodology includes:

- For **name-based queries**, we used a trie data structure that supports efficient retrieval of IDs for exact and prefix-based searches.
- For **year-based queries**, we utilize hashing with the formula $(\text{year} - \text{min_year})$, storing all relevant IDs at the index generated by this formula.
- For **join queries**, we first retrieve matching IDs from the trie based on the name and filter the results to include only those that satisfy the year condition.

The system aims to optimize retrieval speed and memory usage by tailoring indexing methods to the specific characteristics of each query type.

2 Disk Layout Strategy

The disk contains IDs stored twice: once sorted by name (and by year for entries with the same name) and once sorted by year (and by name for entries with the same year). The indices mentioned hereafter refer to the locations on the disk where the IDs are stored. This organization ensures that IDs with similar names are grouped together (sorted by name) and those with closer graduation years are also kept together (sorted by year). This structure helps to minimize seek time, as results with similar name and year values are stored in proximity.

3 Indexing Choices

3.1 Index Types and Justification

3.1.1 Trie Structure for Name Field

A trie was implemented to handle exact and prefix-based queries on names. Each node in the trie stores a character, a boolean flag indicating if the character marks the end of a word, a list of child nodes, and a list of IDs corresponding to names that end at that node.

Insertions: Each word is inserted in the order of tuples sorted by name, with the stored ID being a hashed value combining the index and graduation year, calculated as: $\text{Index Value} \times 10000 + \text{Year}$.

Memory: The maximum depth of the trie is 10, corresponding to the longest word length of 10 characters. The largest list size stored at any node is 88, meaning up to 88 alumni may share the same name in the given dataset. The trie structure is efficient for storage in memory. Memory taken by this structure in worst case is $\text{Total Number of Indices} + 10 \times (\text{Number of Unique Names}) = 100000 + 10 \times 62281 = 722810$, assuming Number of Indices and Number of Unique Names as in public.csv.

Query:

- “ = ” Queries: A maximum of 10 operations is required to reach the leaf node of a query, with an additional $O(\text{output.size})$ operations to retrieve the result. If the exact word is found, the list stored at the end of the word is returned; otherwise, an empty list is returned.
- “LIKE” Queries: We search up to the given characters in nodes of trie, then perform a DFS from the last character to reach the end of each word, as the LIKE queries are restricted to patterns of the form “xyz%”.

The indices for this query search are always returned in sorted order by name, as they are inserted into the trie sequentially in sorted order, and the DFS search also follows this insertion order. Also, for each name, the indices are sorted in increasing order of index value, as this is ensured during the insertion process.

3.1.2 Hashing for Year Field

Storing the index values to the bucket/list given by the hash value (year-min_year) was used for queries involving only year.

Insertions: Each index is inserted based on tuples sorted by year. The year value for each index is used to calculate the corresponding list index using the formula provided earlier, and the index is then stored in the designated list. The lists corresponding to years are stored in ascending order, as the hashed value increases with the progression of the year.

Memory: The memory used is equal to the number of tuples present in the database, as each index is stored only once. In the worst case, the number of lists created is 200, since the range of years is limited to 1900 to 2100.

Query: The given year value is subtracted from the minimum year to obtain an index value for the list, say val . Depending on the comparator, the following algorithm is used:

- “ = ”: The list corresponding to the index val is returned, as it contains all indices associated with the specified graduation year.
- “ >= ”: All lists starting from the index val to the end are returned, as the indices that meet the condition are stored in buckets/lists indexed at values $\geq$ to val .
- “ <= ”: All lists starting from the index 0 to the index val are returned, as the indices that meet the condition are stored in buckets/lists indexed at values $\leq$ to val .

The indices are returned in sorted order by year, and for each year, the indices are also sorted in increasing order of index value, as this is ensured during the insertion process.

3.1.3 Join Operation for Both Name and Year Field

First, the indices for the Name field are retrieved using the trie structure as described earlier, and then the results are filtered based on the specified year condition. The index value and the corresponding year used for condition checking are retrieved from the stored hash value. The returned indices may

be sorted by name or year, depending on which order results in the smallest spread. This approach minimizes seek time and is discussed in the optimization later.

3.2 Index Not Chosen and Justification

- **B+ Tree for Year Field:** Although efficient for range queries, B+ trees are impractical due to memory overhead and high duplication, as shown in Statistics. A sorted list with binary search ($O(\log n) + (\text{output size})$) or hashed buckets ($O(\text{output size})$) provides better efficiency for this dataset.
- **Cross-Join:** This requires a significant number of operations in computing both name based and year based queries, so it is preferable to use the option with fewer records to filter against other conditions, which is true in most cases.

4 Statistics

4.1 Dataset Characteristics

- **Total Entries:** In the data provided we had around 100,000 alumni record which made the task memory heavy.
- **Distinct Years:** We had around 61 unique years in our database with many duplication in the table for the recent year entries. This renders the B+ tree inefficient for handling year queries.
- **Year Distribution:** The year 2020 has the most entries for that year (4162 alumni) as compared to least (21 entries) for the year 1964.
- **Name Length Distribution:** The most common name lengths are 5-6 characters (67,037 entries total in the database with this length), while only a few names reach 9 characters.
- **ID Uniqueness:** We had different IDs for each entry which allowed us to use the direct indexing to map these to the retrieved data.

4.2 Statistical Insights for Building Index

High duplication of years and commonality in name length justify using a trie for names and hashing for years.

Name Queries: The average length of characters in each word is typically around 5 to 6, making the trie structure efficient. This also supports the use of DFS for LIKE queries, which generally requires only 5 to 6 operations.

Year Queries: The spread of years in the actual database is 200 or fewer, allowing for the creation of 200 buckets for each year. In contrast, binary search necessitates sorting through 100,000 tuples. Furthermore, binary search would require storing pairs of (year, id), whereas using hashed index buckets only requires storing the IDs. Thus, hashing based buckets is time as well as memory efficient.

Join Queries: In the provided dataset, the maximum number of entries returned by the trie for a Name query of the form " $x\%$ " (where x represents one or more characters) would be 26,691 which need to be filtered for year condition. In contrast, a year-based query typically yields around 50,000 results on average. This demonstrates that the trie is more time-efficient for handling join queries.

5 Query Optimization Techniques

- **Year-Only Queries:** Year-based queries use year-wise buckets for constant-time retrieval. The order of the indices in the buckets, arranged by increasing year, allows for results to be retrieved in $O(\text{output_size})$, thus incurring no additional time overhead during the search. Before executing the actual query, we check for obvious conditions, such as whether the year is less than the minimum year or greater than the maximum year, and make results based on the provided comparator. For instance, if the year is less than the minimum year and the comparator is "greater than or equal to" ($\geq$), we need to return the entire index list.
- **Name-Only Queries:** Trie-based indexing enables efficient exact and prefix-based retrieval in constant time (a maximum of 10 operations). By storing indices in sorted order and

performing DFS in that same order, there is no sorting overhead, resulting in no additional time required to reduce seek time. Before executing the actual query, we check if the provided string is empty. If the query is prefix-based, we return all indices; if it is for exact retrieval, we return an empty result.

- **Combined Name and Year Queries:** These are optimized by retrieving IDs with the trie and filtering by year, minimizing unnecessary processing of both queries. Before returning the final answer, we examine the corresponding index based on year sorting as well and select the one with the smallest spread. This spread is calculated as the difference between the maximum and minimum index values, allowing us to identify the index that minimizes seek time. For example, suppose the name-based sorting in the trie yields an index order of (0, 1, 3, 4, 5), which corresponds to the indices (8, 10, 12, 16, 18) in the year-based index. The seek time for the first index can be calculated as $(1-0-1)+(3-1-1)+\dots+(5-4-1)$, which is equivalent to $5-0=5$, i.e. (maximum value of the index - minimum value of index). In the second case, the seek time would be equivalent to $18-8=10$. The minimum value in this scenario is 5, so we will opt to use the name-based sorting index. To facilitate this evaluation, we need to maintain an explicit mapping that translates the disk index from name-based sorting to year-based sorting.

5.1 Execution Strategy and Memory Management

Efficient memory handling is achieved by storing indices exactly once in each structure.

- **Trie Optimization:** The leaves of the trie are marked with a boolean value to store the index solely at the leaf nodes. To evaluate the year condition in the Join Query, we do not store the year explicitly; instead, we hash it with the index value to minimize the index size.

6 Conclusion

The selected indexing and optimization strategies efficiently balance memory usage and processing speed. Trie structures for names and hashing for years optimize retrieval, while our join query approach minimizes unnecessary operations. There are further potential optimizations, such as storing only the minimum and maximum index values in each year bucket instead of explicitly storing all indices. Since the indices are sorted by year and entries for the same year are grouped together, this approach would reduce memory usage without affecting retrieval time. However, these optimizations could not be implemented due to time constraints.